

Setup Instructions for SQL Server

1. Download the Project (Clone)

First, you need to pull the code down from GitHub to your machine.

1. Open **Command Prompt** (or PowerShell).
2. Navigate to the folder where you want to save the project (e.g., `cd Documents`).
3. Run the following command to clone the repository:

Bash

```
git clone https://github.com/qsanders/CMS_Data_Governance_Project.git
```

4. Move inside the newly created project folder:

Bash

```
cd CMS_Data_Governance_Project
```

2. Set Up the Python Environment

A virtual environment keeps the project's dependencies completely separate from the rest of your laptop.

1. Create the virtual environment by running:

Bash

```
python -m venv venv
```

2. Activate the virtual environment (this is the Windows command):

Bash

```
venv\Scripts\activate
```

(You should now see `(venv)` at the beginning of your terminal prompt).

3. Install Dependencies & SQL Server Driver

1. Install the base project requirements:

Bash

```
pip install -r requirements.txt
```

2. **Crucial Step for SQL Server:** The original code uses a driver called `psycopg2` to talk to PostgreSQL. Because you are evaluating this on SQL Server, you need to install the Microsoft ODBC driver for Python. Run:

Bash

```
pip install pyodbc
```

4. Create Your Configuration File

You need a `.env` file to hold your local database connection details so the script knows where to load the data.

1. In the root of the `CMS_Data_Governance_Project` folder, create a new file and name it exactly `.env`.
2. Open it in a text editor (like VS Code or Notepad) and add your SQL Server details. If you log into SQL Server using Windows Authentication, you only need the host and database name:

Plaintext

```
DB_HOST=localhost # Or your specific SQL Server instance name, e.g.,  
localhost\SQLEXPRESS  
DB_NAME=cms_data_governance  
DB_USER=your_username # (Optional: Only if using SQL Authentication)  
DB_PASS=your_password # (Optional: Only if using SQL Authentication)
```

5. Update the Python Script for SQL Server

You need to make a one-line change to the Python script so it translates the connection string for SQL Server instead of Postgres.

1. Open `code_scripts/raw_cms_DE_SynPUF_data_loader.py`.
2. Scroll down to the `get_database_engine()` function (around line 50).
3. Find the `url` variable.

If you use Windows Authentication (Trusted Connection) to log into SQL Server, replace the URL line with this:

Python

```
url =  
f"mssql+pyodbc://{db_host}/{db_name}?driver=ODBC+Driver+17+for+SQL+Server&Trusted_Connection=yes"
```

If you use a specific Username and Password to log into SQL Server, replace the URL line with this:

Python

```
url =  
f"mssql+pyodbc://{db_user}:{db_pass}@{db_host}/{db_name}?driver=ODBC+Driver+17+for+SQL+Server"
```

(Note: Ensure you actually have "ODBC Driver 17 for SQL Server" installed on your Windows machine; it is usually installed by default with SQL Server Management Studio).

6. Run the Project

With the environment activated, the driver installed, and the connection string pointing to SQL Server, you are ready to execute the code.

Run the script from your terminal:

Bash

```
python code_scripts/raw_cms_DE_SynPUF_data_pull.py
```